
Object-Oriented Programming (OOP) in Python

Important Instruction for Week 04 Lab:

Do not use AI tools or external code generators to complete these exercises. They are intended for you to practice your own coding abilities. Think through the logic, reference Python syntax, and work through the challenges independently.

Your task is to write a Python Programming Language to complete the exercises below:

1. Create a basic class and instantiate an object.

Tasks:

- Create a class called Car with the attributes: make, model, and year.
- Add a method display_info() that prints the car's details.
- Create an object from the Car class and display its information.

Expected output:

```
Car details:
Make: Toyota
Model: Corolla
Year: 2020
```

2. Learn how to use constructors and simple methods.

Tasks:

- Modify the Car class to include a constructor (__init__ method) that initializes the make, model, and year.
- Add a method is_vintage() that returns True if the car is older than 25 years and False otherwise.
- Create two objects, one vintage and one non-vintage, and call the is_vintage() method on each.

Expected output:

```
Car 1 is vintage: False
Car 2 is vintage: True
```

3. Understand how to add properties and update them.

Tasks:

- Create a class called Student with attributes: name, age, and grade.
- Add a method update_grade(new_grade) that changes the student's grade.
- Create an object, update its grade, and print the updated information.

Expected output:

Before grade update:

Name: John

Age: 20

Grade: B

After grade update:

Name: John

Age: 20

Grade: A

4. Use multiple methods within a class.

Tasks:

- Create a class called BankAccount with attributes: owner and balance.
- Add methods deposit(amount) and withdraw(amount), where the withdraw() method checks if there are sufficient funds before subtracting.
- Create an object, perform some deposits and withdrawals, and print the resulting balance.

Expected output:

Initial balance for John: \$1000

Depositing \$500

New balance: \$1500

Withdrawing \$300

New balance: \$1200

```
Attempting to withdraw $2000
Insufficient funds. Withdrawal failed.
```

```
Final balance: $1200
```

5. Understand how objects can interact with each other.

Tasks:

- Create two classes: Book with attributes title, author, and price, and Library with an attribute books (a list of Book objects).
- Add a method add_book(book) in Library to add Book objects to the library.
- Add another method show_books() that prints the details of each book in the library.
- Create a Library object, add several Book objects to it, and display the book list.

Expected output:

```
Books in the library:
```

1. Title: The Great Man, Author: F. Ane Fit, Price: \$10.99
2. Title: The story about APT, Author: Fake Man, Price: \$12.50
3. Title: 1984, Author: George Orwell, Price: \$9.75

6. Understand how to use lists within a class to store data.

Tasks:

- Create a class called Classroom with attributes: class_name and students (a list of student names).
- Add methods add_student(name) to add a new student and list_students() to print all student names.
- Create an object, add some students, and list them.

Expected output:

```
Added student: Alice
Added student: Bob
Added student: Charlie
Students in Math 101:
- Alice
- Bob
```

- Charlie

7. Learn how to use dictionaries to store key-value pairs in a class.

Tasks:

- Create a class called PhoneBook with an attribute contacts (a dictionary where keys are names and values are phone numbers).
- Add methods add_contact(name, number) to add a contact and find_contact(name) to display a contact's number or a message if not found.
- Create an object, add contacts, and look up a contact.

Expected output:

Added contact: John -> 123-456-7890

Added contact: Jane -> 987-654-3210

John's number: 123-456-7890

Alice not found in phone book.

8. **(Challenge Yourself🌟)** Create a class that allows full CRUD (Create, Read, Update, Delete) operations to manage players and teams within a sports league.

Tasks:

- Create a class called SportsLeague with the following attributes:
 - o teams (a dictionary where keys are team names and values are lists of player dictionaries; each player dictionary has keys 'id', 'name', and 'position')
- Implement the following methods:
 - o add_team(team_name) to add a new team.
 - o add_player(team_name, player_id, player_name, position) to add a player to a specified team.
 - o view_team(team_name) to display all players on a team.
 - o update_player(team_name, player_id, new_name=None, new_position=None) to update a player's information.
 - o remove_player(team_name, player_id) to remove a player from a team.
- Create an instance, add teams and players, view teams, update player information, and remove players.

Tips:

- add_team: Adds a new team if it doesn't already exist.
- add_player: Adds a player to a specified team, checking for duplicate player IDs using the any() function.

- `view_team`: Displays all players in a specified team or notifies if the team has no players.
- `update_player`: Updates a player's name and/or position based on the given `player_id`.
- `remove_player`: Removes a player from a team using `enumerate()` to iterate with an index for removal.

*Note: This exercise helps students practice using classes, dictionaries, and lists, as well as the built-in **any()** and **enumerate()** functions for efficient operations.*

Expected output:

```
Team 'Tigers' added.  
Team 'Sharks' added.  
Player 'John Doe' added to team 'Tigers'.  
Player 'Alice Smith' added to team 'Tigers'.  
Player 'Bob Brown' added to team 'Sharks'.
```

Viewing teams:

```
Team 'Tigers' players:  
ID: 1, Name: John Doe, Position: Forward  
ID: 2, Name: Alice Smith, Position: Goalkeeper  
Team 'Sharks' players:  
ID: 3, Name: Bob Brown, Position: Defender
```

Updating player info:

```
Player ID 1 updated in team 'Tigers'.  
Team 'Tigers' players:  
ID: 1, Name: Johnny Doe, Position: Striker  
ID: 2, Name: Alice Smith, Position: Goalkeeper
```

Removing a player:

```
Player 'Alice Smith' removed from team 'Tigers'.  
Team 'Tigers' players:  
ID: 1, Name: Johnny Doe, Position: Striker
```

Reminder: *Do not use AI tools to generate your solution. Think through how you would logically check each condition and write your code accordingly.*



At the end of this lab, you need to produce one file named: **full_name_W04_Lab.py**

- Example: `Chan_Dara_W04_Lab.py`

🚨 *Note: If you cannot submit the file in .py format, please submit it in a .zip or .txt format with the same filename (e.g., `Chan_Dara_W04_Lab.zip`).*